

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

про виконання лабораторної роботи №1
з курсу «Професійний Python»

«Організація професійного Python-проєкту та Git»

Виконав:
студент 3 курсу, групи ФЕП-31с
Гарбів Данило

Перевірив:
професор
Свелеба Сергій Андрійович

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №1. «Організація професійного Python-проєкту та Git».

3. Мета роботи

Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом `src-layout`, працювати з модулями та пакетами Python, створювати файл `pyproject.toml`, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

4. Номер індивідуального варіанта

Варіант №9: «Облік спортивних результатів».

5. Постановка задачі

Розробити консольний Python-застосунок для обліку, фільтрації та статистичного аналізу результатів спортивних змагань відповідно до індивідуального варіанта №9.

5.1. Вимоги до сутності спортсмена:

- **ім'я** (`name: str`) — ім'я та прізвище спортсмена;
- **вид спорту** (`sport: str`) — спортивна дисципліна;
- **результат** (`result: float`) — кількісний показник (бали, секунди, метри тощо);
- **категорія** (`category: str`) — вікова або кваліфікаційна група («Дорослі», «Юніори», «Майстри»).

5.2. Вимоги до бізнес-логіки:

- додавання нового результату до колекції;
- пошук результатів за видом спорту;
- визначення найкращого результату (за видом спорту або серед усіх);
- обчислення середнього значення результатів;
- формування впорядкованого рейтингу спортсменів.

5.3. Загальні та підвищені вимоги до реалізації:

- Організація структури коду за принципом `src-layout`;
- Опис сутності через `@dataclass` із валідацією полів та власним класом винятку `InvalidAthleteDataError`;
- Розподіл логіки на окремі модулі: `models.py`, `services.py`, `main.py`;
- Повне покриття анотаціями типів (`type hints`);
- Точка входу із захистом через `if __name__ == "__main__":`;

- Стандартизований конфігураційний файл `pyproject.toml` (PEP 621) із підтримкою `pip install -e .`;
- Наявність файлів `.gitignore`, `README.md` та автоматизованих тестів у каталозі `tests/`;
- Фіксація етапів розробки в Git за допомогою логічних атомарних комітів;
- Зручний інтерфейс користувача: консольне інтерактивне меню (`menu-driven interface`), захищене введення з клавіатури, пошук за декількома критеріями одночасно та розрахунок статистики.

6. Структура проєкту

Проект побудовано за принципом `src-layout`:

```
lab1/
├── .gitignore           # Шаблони файлів, які ігноруються системою Git
├── pyproject.toml       # Стандартна конфігурація проєкту та збирання (PEP 621)
├── README.md           # Технічна документація та інструкція користувача
├── REPORT.md           # Повний текст звіту про виконання роботи
├── src/                # Директорія вихідного коду (src-layout)
│   ├── sports_tracker/ # Головний пакет програми
│   │   ├── __init__.py # Ініціалізація пакета, константа версії (__version__)
│   │   ├── models.py   # Модель даних Athlete (@dataclass), винятки валідації
│   │   ├── services.py # Чиста бізнес-логіка (обчислення, фільтри, рейтинг)
│   │   └── main.py     # Точка входу, консольне меню, валідоване введення
└── tests/              # Каталог автоматизованих тестів
    └── test_services.py # Модульні тести функцій бізнес-логіки
```

7. Опис створених модулів

1. `src/sports_tracker/__init__.py` — ініціалізує пакет, містить документаційний рядок та публічну версію програми `__version__ = "0.1.0"`.
2. `src/sports_tracker/models.py` — містить визначення винятку `InvalidAthleteDataError(ValueError)` та модель даних `Athlete` на базі `@dataclass`. Реалізує перевірку непорожності полів та невід'ємності результату в методі `__post_init__`, а також форматове представлення через `display_info`.
3. `src/sports_tracker/services.py` — модуль чистої предметної логіки без операцій введення/виведення:
 - `add_result` — додавання спортсмена до списку;
 - `filter_by_sport` — фільтрація записів за видом спорту (`case-insensitive`);
 - `filter_by_multiple_criteria` — пошук одночасно за спортом і категорією;
 - `find_best_result` — пошук лідера (максимальний або мінімальний показник);

- `calculate_average_result` — розрахунок середнього значення з округленням;
- `generate_ranking` — сортування колекції спортсменів за результатом;
- `calculate_statistics` — розрахунок зведених показників (`count`, `avg`, `min`, `max`).

4. **`src/sports_tracker/main.py`** — модуль взаємодії з користувачем: реалізує функції безпечного введення з клавіатури (`prompt_non_empty_string`, `prompt_positive_float`), консольне меню (пункти 0–7), демонстраційні дані та автоматичний режим `--demo`.
5. **`tests/test_services.py`** — автоматизовані модульні тести для перевірки всіх функцій сервісів та валідації в моделях.

8. Текст основних фрагментів коду

8.1. Модуль моделей даних: `src/sports_tracker/models.py`

```
from dataclasses import dataclass

class InvalidAthleteDataError(ValueError):
    """Raised when athlete data fails validation."""
    pass

@dataclass
class Athlete:
    """Represents an athlete and their performance result."""
    name: str
    sport: str
    result: float
    category: str

    def __post_init__(self) -> None:
        self.name = self.name.strip()
        self.sport = self.sport.strip()
        self.category = self.category.strip()

        if not self.name:
            raise InvalidAthleteDataError("Athlete name cannot be empty.")
        if not self.sport:
            raise InvalidAthleteDataError("Sport name cannot be empty.")
        if not self.category:
            raise InvalidAthleteDataError("Category cannot be empty.")
        if self.result < 0:
            raise InvalidAthleteDataError(
                f"Result cannot be negative. Received: {self.result}"
            )

    @property
    def display_info(self) -> str:
        return f"{self.name} | {self.sport} | {self.result:.2f} | {self.category}"
```

8.2. Модуль бізнес-логіки: `src/sports_tracker/services.py`

```
from sports_tracker.models import Athlete

DEFAULT_PRECISION: int = 2

def add_result(athletes: list[Athlete], athlete: Athlete) -> None:
    athletes.append(athlete)

def filter_by_sport(athletes: list[Athlete], sport: str) -> list[Athlete]:
    target = sport.strip().lower()
    return [a for a in athletes if a.sport.lower() == target]

def filter_by_multiple_criteria(
    athletes: list[Athlete],
    sport: str | None = None,
    category: str | None = None,
) -> list[Athlete]:
    result = athletes
    if sport is not None and sport.strip():
        target_sport = sport.strip().lower()
        result = [a for a in result if a.sport.lower() == target_sport]
    if category is not None and category.strip():
        target_category = category.strip().lower()
        result = [a for a in result if a.category.lower() == target_category]
    return result

def find_best_result(
    athletes: list[Athlete],
    sport: str | None = None,
    higher_is_better: bool = True,
) -> Athlete | None:
    candidates = filter_by_sport(athletes, sport) if sport else athletes
    if not candidates:
        return None
    return max(candidates, key=lambda a: a.result) if higher_is_better else min(candidates, key=lambda a: a.result)

def calculate_average_result(
    athletes: list[Athlete],
    sport: str | None = None,
) -> float:
    candidates = filter_by_sport(athletes, sport) if sport else athletes
    if not candidates:
        return 0.0
    return round(sum(a.result for a in candidates) / len(candidates), DEFAULT_PRECISION)

def generate_ranking(
    athletes: list[Athlete],
    sport: str | None = None,
    descending: bool = True,
) -> list[Athlete]:
    candidates = filter_by_sport(athletes, sport) if sport else athletes
    return sorted(candidates, key=lambda a: a.result, reverse=descending)

def calculate_statistics(
    athletes: list[Athlete],
    sport: str | None = None,
) -> dict[str, float | int]:
    candidates = filter_by_sport(athletes, sport) if sport else athletes
    if not candidates:
        return {"count": 0, "average": 0.0, "min_result": 0.0, "max_result": 0.0}
```

```

results = [a.result for a in candidates]
return {
    "count": len(candidates),
    "average": round(sum(results) / len(results), DEFAULT_PRECISION),
    "min_result": min(results),
    "max_result": max(results),
}

```

8.3. Головний модуль застосунку: `src/sports_tracker/main.py` (фрагмент)

```

def print_menu() -> None:
    print("\n" + "=" * 50)
    print("        СИСТЕМА ОБЛІКУ СПОРТИВНИХ РЕЗУЛЬТАТІВ        ")
    print("=" * 50)
    print("1. Показати всі спортивні результати")
    print("2. Додати новий результат (введення з клавіатури)")
    print("3. Пошук результатів за видом спорту")
    print("4. Пошук за декількома критеріями (спорт + категорія)")
    print("5. Визначити найкращий результат")
    print("6. Розрахувати середній результат та статистику")
    print("7. Сформувати рейтинг спортсменів")
    print("0. Вихід із програми")
    print("=" * 50)

def run_menu(athletes: list[Athlete]) -> None:
    while True:
        print_menu()
        choice = input("Оберіть пункт меню (0-7): ").strip()
        if choice == "1":
            print_athletes(athletes, "Усі зареєстровані результати")
        elif choice == "2":
            handle_add_athlete(athletes)
        elif choice == "3":
            handle_filter_by_sport(athletes)
        elif choice == "4":
            handle_multi_search(athletes)
        elif choice == "5":
            handle_best_result(athletes)
        elif choice == "6":
            handle_statistics(athletes)
        elif choice == "7":
            handle_ranking(athletes)
        elif choice == "0":
            print("\nДякуємо за використання програми. До побачення!")
            break

def main() -> None:
    athletes = create_demo_athletes()
    if len(sys.argv) > 1 and sys.argv[1] == "--demo":
        run_demo_scenario(athletes)
        return
    run_menu(athletes)

if __name__ == "__main__":
    main()

```

9. Пояснення `pyproject.toml`

Файл `pyproject.toml` є уніфікованим декларативним стандартом для керування конфігурацією та процесом побудови Python-пакетів (специфікації PEP 517, PEP 518 та PEP 621).

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "sports-tracker"
version = "0.1.0"
description = "Sports results tracking system - Laboratory work 1 for Professional Python"
readme = "README.md"
requires-python = ">=3.11"
authors = [
    { name = "Danylo Harbich" }
]
keywords = ["sports", "tracker", "python", "laboratory"]
classifiers = [
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
]

[project.scripts]
sports-tracker = "sports_tracker.main:main"
```

Детальний опис компонентів:

- **[build-system]** — задає інструменти, необхідні для білду проєкту (`setuptools>=70`) та бекенд-інтерфейс (`setuptools.build_meta`), що дозволяє менеджеру `pip` зібрати `wheel`-пакет.
- **[project]** — містить метадані: унікальне ім'я дистрибутива (`sports-tracker`), початкову версію (`0.1.0`), файл документації (`README.md`), версійні обмеження інтерпретатора (`>=3.11`) та дані автора.
- **[project.scripts]** — створює виконувану команду консолі `sports-tracker`, що викликає `sports_tracker.main:main`. Завдяки цьому після інсталяції через `pip install -e .` користувач може запускати застосунок простою командою `sports-tracker` з будь-якого місця.

10. Вміст .gitignore

```
# Byte-compiled / optimized / DLL files
__pycache__/
*.py[cod]
*$py.class

# Distribution / packaging
build/
dist/
*.egg-info/

# Virtual environments
.venv/
venv/

# IDE and editor settings
.idea/
.vscode/

# Testing and coverage
.pytest_cache/
.coverage

# Environment variables
.env
```

Пояснення: файл запобігає потраплянню до репозиторію кешів компілятора (`__pycache__`), платформозалежного віртуального оточення (`.venv/`), індивідуальних налаштувань IDE розробника (`.vscode/`, `.idea/`) та тимчасових каталогів дистрибуції.

11. Фрагмент README.md

```
# Sports Tracker (Облік спортивних результатів)
Навчальний проєкт з дисципліни «Професійний Python» (Лабораторна робота №1).
Індивідуальний варіант №9: «Облік спортивних результатів».

## Встановлення
1. python -m venv .venv
2. .venv\Scripts\activate (Windows) або source .venv/bin/activate (Linux/macOS)
3. python -m pip install -e .

## Запуск
- Через модуль: python -m sports_tracker.main
- Через команду: sports-tracker
- Демонстраційний режим: sports-tracker --demo
- Тести: python tests/test_services.py
```

12. Скріншоти та результати запуску застосунку

12.1. Головне меню програми (інтерактивний режим)

Ласкаво просимо до Системи обліку спортивних результатів!
Лабораторна робота №1 з дисципліни 'Професійний Python'.

```
=====
СИСТЕМА ОБЛІКУ СПОРТИВНИХ РЕЗУЛЬТАТІВ
=====
1. Показати всі спортивні результати
2. Додати новий результат (введення з клавіатури)
3. Пошук результатів за видом спорту
4. Пошук за декількома критеріями (спорт + категорія)
5. Визначити найкращий результат
6. Розрахувати середній результат та статистику
7. Сформувати рейтинг спортсменів
0. Вихід із програми
=====
Оберіть пункт меню (0-7):
```

12.2. Відображення таблиці спортсменів (Пункт 1)

=== Усі зареєстровані результати ===

№	Ім'я спортсмена	Вид спорту	Результат	Категорія
1	Ярослава Магучіх	Стрибки у висоту	2.10	Дорослі
2	Ірина Геращенко	Стрибки у висоту	2.00	Дорослі
3	Юлія Левченко	Стрибки у висоту	1.95	Дорослі
4	Олег Верняєв	Спортивна гімнастика	15.30	Майстри
5	Ілля Ковтун	Спортивна гімнастика	15.65	Юніори
6	Михайло Романчук	Плавання	880.20	Дорослі
7	Олександр Желтяков	Плавання	895.10	Юніори

12.3. Додавання результату з клавіатури з валідацією (Пункт 2)

```
--- Додавання нового спортивного результату ---
Введіть ім'я та прізвище спортсмена: Марина Бех-Романчук
Введіть вид спорту: Стрибки у довжину
Введіть числовий результат (очки / метри / бали / секунди): 6.93
Введіть категорію (напр. Дорослі, Юніори, Майстри): Дорослі
Успішно додано: Марина Бех-Романчук (Стрибки у довжину)!
```

12.4. Пошук результатів за видом спорту (Пункт 3)

Введіть вид спорту для пошуку: Стрибки у висоту

=== Результати за видом спорту: 'Стрибки у висоту' ===

№	Ім'я спортсмена	Вид спорту	Результат	Категорія
1	Ярослава Магучіх	Стрибки у висоту	2.10	Дорослі
2	Ірина Геращенко	Стрибки у висоту	2.00	Дорослі
3	Юлія Левченко	Стрибки у висоту	1.95	Дорослі

12.5. Визначення найкращого результату (Пункт 5)

Введіть вид спорту (або натисніть Enter для пошуку серед усіх): Стрибки у висоту

Найкращий результат у спорті 'Стрибки у висоту':

Спортсмен: Ярослава Магучіх
Вид спорту: Стрибки у висоту
Результат: 2.10
Категорія: Дорослі

12.6. Розрахунок середнього результату та статистики (Пункт 6)

Введіть вид спорту (або Enter для загальної статистики): Стрибки у висоту

Статистичні показники (для спорту 'Стрибки у висоту'):

Кількість учасників: 3
Середній результат: 2.02
Мінімальний показник: 1.95
Максимальний показник: 2.10

12.7. Формування та сортування рейтингу (Пункт 7)

Введіть вид спорту для рейтингу (або Enter для загального): Стрибки у висоту

=== Рейтинг спортсменів (Стрибки у висоту) ===

№	Ім'я спортсмена	Вид спорту	Результат	Категорія
1	Ярослава Магучіх	Стрибки у висоту	2.10	Дорослі
2	Ірина Геращенко	Стрибки у висоту	2.00	Дорослі
3	Юлія Левченко	Стрибки у висоту	1.95	Дорослі

13. Результат виконання команди `git status`

```
$ git status
On branch main
nothing to commit, working tree clean
```

14. Результат виконання команди `git log --oneline`

```
$ git log --oneline
ad1ebbc Finalize laboratory report
28fd2c9 Add laboratory report documentation
0f967d1 Add application entry point and documentation
8d36b71 Add athlete model and business logic
1603aea Create project structure
```

15. Відповіді на контрольні запитання

1. Що таке Python-проект?

Python-проект — це організована сукупність файлів вихідного коду, пакетів, конфігураційних файлів, документації, автоматизованих тестів та списку залежностей, що об'єднані для вирішення конкретної задачі як самостійна програмна система чи бібліотека.

2. Чим проєкт відрізняється від одного Python script?

Скрипт — це один автономний .py-файл з лінійними інструкціями для локального виконання. Проєкт має модульну архітектуру, систему керування залежностями, ізольоване оточення (.venv), стандартну систему збірки (pyproject.toml), контроль версій (Git) та можливість пакування й поширення через інструменти на кшталт pip.

3. Що таке Python module?

Python-модуль — це файл із розширенням .py, який містить визначення функцій, класів або змінних і призначений для повторного використання в інших модулях через механізм імпортування (import).

4. Що таке Python package?

Python-пакет — це каталог, що містить один або більше модулів (або вкладених пакетів) та службовий файл __init__.py, який забезпечує ієрархічне структурування простору імен.

5. Для чого потрібний __init__.py?

Файл __init__.py сигналізує інтерпретатору Python, що даний каталог є пакетом. Він виконується під час імпорту пакета і дозволяє задавати версію (__version__), налаштовувати скорочені імпорти та керувати публічним інтерфейсом через список __all__.

6. Що таке src-layout?

src-layout — це стандарт структури проєкту, за якого весь вихідний програмний код розташовується у спеціальній піддиректорії src/ (наприклад, src/sports_tracker/), а не безпосередньо в корені репозиторію поруч із конфігураційними файлами.

7. Які переваги має src-layout?

Головна перевага — унеможливлення випадкового імпортування локального незібраного коду при тестуванні. Тести запускаються проти встановленого у віртуальне середовище пакета, що відтворює реальні умови кінцевого користувача. Також це запобігає появі зайвих артефактів (__pycache__) у кореневій папці проєкту.

8. Для чого використовується virtual environment?

Віртуальне середовище забезпечує ізоляцію бібліотек та їхніх версій для конкретного проєкту. Воно гарантує, що встановлення чи оновлення пакетів в одному проєкті не порушить працездатність інших проєктів або системного інтерпретатора Python.

9. Як створити .venv?

За допомогою стандартного вбудованого модуля venv: `python -m venv .venv`.

10. Як активувати virtual environment?

У Windows (PowerShell): `.venv\Scripts\activate`; у Windows (CMD): `.venv\Scripts\activate.bat`; у Linux/macOS: `source .venv/bin/activate`.

11. Для чого використовується pip?

pip — це стандартний менеджер пакетів для Python, призначений для встановлення, оновлення, видалення та інспектування сторонніх бібліотек з репозиторію PyPI або локальних дистрибутивів.

12. Чому рекомендується python -m pip?

Виклик `python -m pip` явно запускає модуль pip, асоційований саме з поточним активним інтерпретатором python. Це виключає ризик звернення до глобального системного pip або іншої версії Python, прописаної у системній змінній PATH.

13. Для чого потрібний pyproject.toml?

pyproject.toml — це єдиний сучасний декларативний конфігураційний файл проєкту (PEP 517, 518, 621), що визначає систему збирання, метадані (назву, версію, автора, ліцензію, залежності) та параметри літерів, форматтерів і тестових фреймворків.

14. Що містить секція [project]?

Секція [project] стандартизує метадані пакета: назву (name), версію (version), короткий опис (description), файл опису (readme), сумісні версії Python (requires-python), список залежностей (dependencies), а також точки входу для консольних команд ([project.scripts]).

15. Що таке editable installation?

Встановлення в режимі редагування (`pip install -e .` або `--editable`). Замість копіювання файлів до site-packages, у віртуальному середовищі створюється посилання на робочий каталог src/. Будь-які зміни у коді відразу стають доступними без повторного виконання інсталяції.

16. Для чого використовується if __name__ == "__main__":?

Конструкція перевіряє, чи файл запущено як основну програму (`__name__ == "__main__"`), чи він імпортований як модуль іншим скриптом. Це дозволяє уникнути автоматичного виконання виконуваного коду або запуску консольного інтерфейсу під час імпорту функцій для тестування.

17. Що таке Git?

Git — це розподілена система керування версіями файлів, яка фіксує історію змін проєкту, дає змогу повертатися до будь-якого попереднього стану коду, вести розробку в окремих гілках та ефективно співпрацювати над кодом у команді.

18. Що таке repository?

Репозиторій — це спеціальне сховище Git (каталог .git), де зберігається повна історія комітів, гілок, тегів, конфігурацій та метаданих, що описують усі зміни проєкту.

19. Що таке commit?

Коміт — це збережений у базі даних Git моментальний знімок (снэпшот) стану проіндексованих файлів проєкту в певний момент часу. Кожен коміт має унікальний хеш-ідентифікатор, дату, автора та змістовне повідомлення про внесені зміни.

20. Для чого використовується git status?

Команда `git status` відображає поточний стан робочого каталогу та індексу (staging area): які файли модифіковано, які підготовлено до коміту, а які не відстежуються системою контролю версій.

21. Для чого git add?

Команда `git add` додає вказані файли чи зміни з робочої директорії до області індексації (staging area), формуючи набір змін для наступного коміту.

22. Для чого git commit?

Команда `git commit` остаточно записує підготовлені в індексі зміни до історії репозиторію, створюючи новий коміт із зазначеним повідомленням (-m).

23. Що показує git log?

Команда `git log` виводить хронологічну історію комітів поточної гілки. Прапорець `--oneline` дозволяє переглядати історію у компактному однорядковому форматі: скорочений хеш і тема повідомлення.

24. Для чого потрібен .gitignore?

Файл `.gitignore` містить шаблони імен файлів та директорій, які Git повинен ігнорувати. Це запобігає потраплянню до репозиторію кешів, бінарних файлів, конфіденційних даних (`.env`), файлів середовищ (`.venv/`) та службових каталогів IDE.

25. Чому .venv не потрібно комітити?

Каталог `.venv` містить велику кількість платформозалежних бінарних файлів і бібліотек, прив'язаних до конкретної операційної системи та шляхів на диску комп'ютера розробника. На іншій машині таке середовище не працюватиме. Замість коміту `.venv` у репозиторій додають декларативні файли залежностей (`pyproject.toml`), що дозволяє відтворити оточення з нуля на будь-якій ОС.

26. Для чого потрібний README?

`README.md` — це основний документ проєкту, який знайомить розробників і користувачів із його призначенням, архітектурою, системними вимогами, інструкціями зі встановлення, запуску, тестування та прикладами використання.

27. Що таке type hints?

Анотації типів (type hints) — синтаксичний механізм мови Python (PEP 484), що дозволяє вказувати очікувані типи аргументів функцій, типів повернення та атрибутів. Вони полегшують читання коду, покращують автодоповнення в IDE та дозволяють виконувати статичний аналіз коду інструментами на зразок `mypy`.

28. Для чого програму розділяють на modules?

Розділення програми на модулі забезпечує принцип слабкої зв'язаності (low coupling) та високої функціональної цілісності (high cohesion). Модульність спрощує навігацію кодовою базою, полегшує внесення змін, усуває дублювання коду та дозволяє ізольовано тестувати окремі підсистеми.

29. Що таке business logic?

Бізнес-логіка (business logic) — це ядро застосунку, яке інкапсулює правила обробки даних, розрахунки, алгоритми фільтрації та обмеження предметної області, залишаючись абсолютно незалежним від користувацького інтерфейсу чи джерел збереження даних.

30. Чому потрібно відокремлювати application entry point від business logic?

Відокремлення точки входу (UI/CLI/API) від бізнес-логіки реалізує архітектурний принцип єдиної відповідальності (Single Responsibility Principle). Завдяки цьому функції логіки стають детермінованими («чистими»), їх легко тестувати автоматичними тестами без імітації консольного вводу (input()), а також це дає змогу змінити інтерфейс програми (наприклад, створити web-версію чи REST API) без модифікації обчислювального ядра.

16. Висновок

Під час виконання лабораторної роботи №1 було детально опановано та практично закріплено стандарти сучасної розробки професійних Python-проектів:

1. **Архітектура та структура:** успішно реалізовано стандартну структуру `src-layout`, що ізолює вихідний код пакету `sports_tracker` від допоміжних конфігураційних файлів і тестів.
2. **Модульний дизайн:** програмну систему розподілено на три основні компоненти:
 - модель даних на базі `@dataclass` із вбудованою валідацією та власним класом винятку `InvalidAthleteDataError` (`models.py`);
 - чиста бізнес-логіка для обліку результатів, пошуку, визначення рекордів, розрахунку середнього значення, статистики та рейтингування (`services.py`);
 - зручна консольна точка входу з меню (`menu-driven interface`) та захищеним введенням даних із клавіатури (`main.py`).
3. **Стандартизація та пакування:** налаштовано декларативний файл `pyproject.toml` за специфікацією PEP 621. Проєкт встановлено у віртуальне середовище `.venv` в режимі редагування (`editable installation`), що забезпечило доступність прямої консольної команди `sports-tracker`.
4. **Контроль версій:** налаштовано коректні правила ігнорування артефактів у `.gitignore`. Життєвий цикл розробки зафіксовано в системі Git за допомогою атомарних логічних комітів із дотриманням загальноприйнятих конвенцій іменування.
5. **Тестування:** розроблено набір юніт-тестів у каталозі `tests/`, що підтвердили стовідсоткову коректність роботи всіх розроблених алгоритмів.

Створений проєкт слугує надійним, гнучким і тестованим базисом для подальшого розширення в наступних лабораторних роботах навчального курсу.